Authority Preservation Across Multi-Principal Agent Delegation

OpenBot

***Abstract*—An agent may work with resources owned by one person, act for another person, use a restricted token, and pass work through several bots. A restart or an old approval must not merge these identities. This design and analysis paper defines an action ceiling from current member, token, platform, context, and bot restrictions. It separates this ceiling from approval. It specifies two role guards: one compares effective permissions, while the other checks each positive rule's full scope. Worked counterexamples show why the second guard is conservative and why principal-relative roles differ from fixed-resource delegation. A proposed admission protocol preserves identity across restart and validates policy versions before admitting a send. Conditional proofs distinguish revocation before local admission from control over a remote provider. A finite executable model checks permission intersections, the cost of conservative role checks, and all placements of one revocation around a read, admission, and send. The model is separate from product tests. The paper gives a reproducible study and an extension plan; it makes no performance or deployment claim.**

***Index Terms*—Agent delegation, Access control, Revocation, Durable execution**

I. Introduction

A useful agent can use tools, delegate tasks, wait for a person, and continue after a server stops. Whose rights does its next action use? The chat owner, bot owner, and approver may all differ from the caller. Treating any one as a general identity can give a small request much wider rights.

Consider a channel whose chat belongs to its administrator. A speaker asks its bot to delegate work. The helper waits for approval and resumes on another node. The administrator and helper's connection may have broad access; the speaker may have narrow access. A fourth person may approve the action. These relationships do not grant the speaker their combined rights.

This is a form of the confused deputy problem: a program uses its own rights for a request that should use the caller's rights [1]. Agents add long waits, changing policies, and generated tool choices. A language model may select an action, but a deterministic check decides whether the identified caller may cause it. This paper studies that boundary, rather than natural-language rule classification or shared-memory privacy.

The work makes three contributions. First, it defines a ceiling for each concrete action and keeps the resource owner, actor, original token, bot, and delegation path distinct. Second, it compares two fully specified role guards. One checks the effective allowed set; the other checks every positive rule scope, even when another rule hides it. Third, it gives a version-checked admission protocol with separate rules for replay and a new send. These rules make the revocation boundary explicit.

The results are a specified protocol, conditional proof arguments, and a finite executable study. The protocol is a proposed design, not a claim that all product paths use version-checked admission. A correct access function does not establish that every tool uses it. Complete placement of checks remains a stated assumption and a separate product evaluation target. The contribution is the joint contract across role creation, saved identity, replay, and local admission. Set intersection and version validation are established techniques.

II. Related Work and Scope

Saltzer and Schroeder state least privilege and complete mediation as design principles [2]. Least privilege limits a program's rights. Complete mediation requires a check for each access. Resumed work adds a time boundary: rights valid when a task started may be invalid when it sends. This paper defines checks at those boundaries.

Hardy's confused deputy example separates naming a resource from having authority to use it [1]. Here, a bot can know a connection identifier without the actor having access. Storage ownership helps locate a chat; it does not replace the speaker's authority.

Lampson, Abadi, Burrows, and Wobber develop distributed authentication around principals and statements about authority [3]. Abadi, Burrows, Lampson, and Plotkin give a logic for principals, roles, and delegation [4]. Those works separate who speaks from whom they speak for. Our narrower model adds explicit token state and a mode for recording old results while refusing new actions.

Macaroons support restricted delegation through caveats, which add conditions to a credential [5]. Their restriction model is close to the set intersection used below. A child can narrow a grant without receiving power to widen it. Our proposed protocol reads mutable policy and token state for each send admission. A macaroon could carry a task restriction, but carrying a credential does not remove the need to define freshness, revocation, or the receiver of a generated request.

OAuth 2.0 names the resource owner and client as distinct roles [6]. OAuth token exchange further distinguishes the subject of a token from an actor acting for that subject [7]. These names help explain why a single user field is insufficient. A token exchange format does not, by itself, prove that every exchanged token has no more rights than the incoming token. That remains an authorization rule. Here the original token reference stays in the execution identity so a restart cannot silently fall back to a broader member session.

Park and Sandhu's usage control model includes continuing decisions and mutable attributes [8]. This is a useful comparison for work that waits while rights change. Our design chooses discrete checks at control points rather than claiming continuous control of a remote service. Once a provider has accepted an action, a local policy change may not undo it. That limit is central to the stated guarantees.

Zanzibar uses coherent snapshots and consistency tokens to respect the order of access changes and content changes [9]. It is a closer system comparison than a credential format alone. Our epoch pair checks for any relevant change between a policy read and a local call admission. It is not a replacement for Zanzibar's cross-service consistency design. Nor does a consistency token prove that an agent kept its source actor through a restart. These are separate obligations.

Cedar provides a formal policy language and a sound, complete symbolic encoding for policy analysis [10]. It makes policy containment a stronger baseline than testing a few examples. Our positive-leaf guard chooses a different contract: it can refuse a safe final policy to prevent a hidden broad grant. The comparison is about that contract, not a claim of a new general containment method. Cedar's development work also separates model proofs from tests that compare production code with the model [11]. We use the same distinction when reporting our finite model evidence; our proofs are written arguments, not machine-checked proofs.

III. Problem Model

A. Identity and actions

A principal is an entity whose identity affects an access decision. An action is a tuple of an operation, a resource, and relevant parameters. A resource has a kind, an identifier, an organization, and any fields used by policy. Parameters matter when a restriction covers, for example, a message destination or an operation argument. Let U be the set of all well-formed actions under a fixed resource schema.

An execution identity is written $I = (o, a, k, b, d, e)$. Here o is the resource owner, a the actor, k the original token when one exists, b the current bot, d the delegation record, and e the parent execution. A bot is a configured agent worker; a chat is a stored conversation. A channel connects a shared outside conversation to one chat. Identity is trusted stored input, not text supplied by a model.

TABLE I
Different identity parts answer different questions. A resource locator is not an authority grant.

Identity part	**Meaning in a check**
Owner	Locates the resource and its account context.
Actor	Supplies the member rights for this work.
Original token	Narrows the actor's rights and keeps expiry.
Bot	Names the worker and its action restrictions.
Delegation	Binds the child to the accepted task.
Parent execution	Allows a check of the stored source path.

The owner and actor can differ. In a channel, the owner holds the chat, while the current speaker is the actor. The authenticated channel mapping identifies that speaker. The bot owner, machine account, and most recent approver are separate principals. None may replace the actor merely because its identifier is easier to obtain.

B. The deterministic ceiling

At time t, let $M_t(a)$ be the active member's allowed actions and $T_t(k)$ the original token's allowed actions. A member session has no token restriction, so its token set is U. A referenced token that is missing or expired gives the empty set. Let P_t exclude platform denials, $N(I)$ restrict the resource and organization context, and $L_t(I)$ restrict the current source and delegation path. Finally, let $B_t(I)$ contain actions allowed by the bot's restrictions.

$$C_t(I) = M_t(a) \cap T_t(k) \cap P_t \cap N(I) \cap L_t(I) \cap B_t(I). \quad (1)$$

Each set can be represented by a predicate. An unavailable check does not give permission. A policy with no positive answer does not give permission. Resource checks and bot capability checks may live at different layers, but every new action must pass all required predicates.

The context term is deliberately not the owner's general access set. Requiring a channel speaker to have broad access to the owner's chat would defeat the channel design. Giving the speaker the owner's rights would be worse. Instead, the channel relation permits a specific way to submit and control work. Ordinary reads of other chats still require the ordinary chat rights.

Approval is an additional condition $Q_t(I, x)$ for action x. A send is permitted only if $x \in C_t(I)$ and $Q_t(I, x)$ holds. An approval can remove a workflow wait. It cannot add an element to $C_t(I)$. Any process that intends to grant new rights must use an explicit policy or capability change and its own grant checks.

C. Assumptions and adversary

The adversary can choose tool inputs, ask a helper to act, keep old task references, and race policy changes with queued work. It can cause a process to stop and can request retries. It cannot modify trusted records outside the public access rules or forge authenticated caller identities. We assume correct resource decoding, integrity of stored parent links, and a sound policy evaluator. We also assume that every external action passes the intended host boundary. These are explicit premises, not results of this paper.

The storage service gives atomic transactions, durable committed records, and reads from one coherent snapshot. Processes may stop at any instruction. Messages may be delayed, lost, or repeated. A committed provider effect can outlive the worker that requested it. A database outage or an unknown policy answer blocks new admission. The protocol does not assume a transaction shared by the database and provider.

The model covers authority within one organization context. Copying a bot configuration into another organization copies no member rights. Cross-organization grants need a separate

trust rule and are outside this model. Rights are checked against immutable action parameters; changing a destination after checking it is a different action and requires a new check.

IV. Exact Scope Checks for New Policies

We define two role guards over the same policy language. Each policy returns allow, deny, or no answer for a concrete action and principal. A matching allow or deny leaf returns its named answer; a nonmatching leaf returns no answer. Conditions may compare resource fields with constants or principal fields, combine Boolean tests, or restrict a resource path. Missing principal fields do not match. Unknown fields, kinds, and actions are rejected before evaluation.

A rule list uses deny priority: any matching deny wins, otherwise a matching allow wins. Conjunction denies if any child denies and allows only if all children allow. An alternative allows if any child allows, otherwise denies if any child denies. An ordered choice returns its first answer. A ceiling returns deny when its cap denies and otherwise returns its base policy's answer. Empty lists and combinators give no answer, as do all remaining cases. There are no implicit allows or policy-complement grants. Member roles and the direct policy combine with deny priority inside the member's organization. The token must independently allow; platform denies always win.

The **effective-set guard** accepts a candidate precisely when its allowed set, under the writer's principal binding, is contained in the writer's current allowed set. The **positive-scope guard** instead visits every allow leaf, including leaves in unused branches and ceiling caps. Wildcards expand over a fixed, versioned resource catalog. Non-organization kinds have empty member scope. These are defined baseline algorithms; neither is asserted to be a deployed system.

For allow leaf r, kind j, and operation v, let $S(r, j, v, a)$ contain matching resource and parameter values inside the caller's organization, with a as principal. The positive-scope guard uses one coherent caller-policy snapshot. A coverage oracle returns `All` only if every value is allowed. Acceptance requires a sound proof of emptiness or `All`; partial and unknown answers are refused. Finite fixtures use enumeration. A symbolic implementation must establish soundness for both answers.

```
own := read coherent caller policy
for each allow leaf r in candidate:
  for each known kind and action in r:
    S := organization(a) AND condition(r, a)
    if S is proved empty: continue
    if coverage(own, S) is not All: reject
accept
```

Listing 1. The defined positive-scope guard checks every allow leaf under one fixed principal binding.

Accepting a role also needs the right to change roles. The authority snapshot and the role write must share version validation; otherwise revocation can race the grant just as it can race a tool send.

The positive-scope guard is conservative about final policy meaning. Suppose a caller can read only chat A. A candidate rule list allows reads of A and B but denies B. Deny priority makes its effective allowed set exactly A. The effective-set guard accepts it. The positive-scope guard rejects its broad allow leaf because B is outside the caller's rights. An allow inside an unused ordered branch is treated in the same way. This can refuse a safe policy, but it avoids depending on a deny or branch order to keep a broad positive grant harmless.

An empty scope is different. A rule confined to another organization gives no member rights, so there is nothing to cover. Changing the resource catalog can change wildcard meaning; a new schema requires a new check.

There is also a principal-binding limit. Suppose Bob can read his own chats and writes a role whose condition means "the principal's own chats." The guard evaluates that condition with Bob as principal and can accept it. If Carol later receives the role, the same condition refers to Carol's chats. The recipient's concrete resource set need not be a subset of Bob's concrete set. This is an intended kind of reusable role template, not proof of fixed-resource delegation.

Thus two claims remain separate. Under a fixed principal binding and sound coverage, the guard bounds each positive scope by the writer's rights. Under rebinding, it checks a policy template in the writer's context. Concrete authority transfer from Bob to Carol requires fixed resources or a proof over permitted rebinding. Neither guard establishes that stronger claim. The proposed agent protocol retains the original actor through a delegated path.

V. Preserving Authority Through Time

A. Admission and delegation

The proposed input rule requires active membership for owner and actor, a valid original token when present, and agreement between chat and identity. Direct input requires permission to run the chat and delegate to its bot. Channel input requires permission to use the enabled channel; its configured owner and bot must match. The owner must still be allowed to use that bot. This route grants no general access to the owner's chats.

A task must name an open delegation whose child chat, bot, owner, actor, token, and saved parent call match. The verifier follows parent records to an accepted input route and rejects missing links or cycles. A review must name a pending, unexpired request assigned to the current reviewer and linked to that source path. It need not give the requester general access to the reviewer. Child creation stores the matched call parameters and identity atomically with the parent link.

For an explicit task restriction H_i, require $C_t(I_{i+1}) \subseteq C_t(I_i) \cap H_i$. The protocol enforces this by checking all retained path restrictions at send admission. A child may change the worker bot but not the original actor or token. Its new restrictions are added by intersection. The retained path includes every ancestor bot restriction that bounded the task, not only the current helper's restrictions. For a path of depth h, write its path term as

$$L_t = L_t^{\text{route}} \cap \left(\bigcap_{i=0}^{h} H_i \right) \cap \left(\bigcap_{i=0}^{h-1} B_t(I_i) \right). \tag{2}$$

Set $H_0 = U$ at the root. The empty ancestor intersection at the root is also U. Removing a parent record or changing its accepted identity invalidates the path; it does not shorten the list of restrictions.

Current-rights preservation limits each send by the original actor's rights at that time. This is the proposed default; a later grant may enable a later action. A fixed-ceiling variant also retains the initial action set. It must capture that set, or freeze the bindings and attributes of its defining predicate. Copying a policy expression that reads changing resource attributes does not freeze its allowed set.

B. Resume, replay, and new sends

A restart reconstructs work from stored identity and the saved tool frame. It must not reconstruct identity from the current chat author, the node account, or the bot owner. A new attempt starts with fresh capability answers. Otherwise a successful answer cached before a revocation could outlive the right that supported it.

Replay records a known result without calling the provider again. Advance permits a new action. A review that resolves its request closes its right to advance but may still need to record that result. Replay cannot change parameters, reopen the review, or send again. Current identity checks still reject a revoked original token.

Define the **fresh-check baseline** as follows. It checks replay access before returning a saved result. For a new send it reads current policies, checks advance access and approval, and checks the current rights of any saved retry decision maker. It then commits a sending marker after checking worker generation and cancellation. The provider call follows that commit. Policy reads and the sending transaction are separate. This exact separation creates the race analyzed below.

Requesting a retry, having current authority, and contacting a provider are separate facts. A saved stop needs no new send and can take effect after rights are removed. A lost reply remains unknown; it does not grant a repeat write.

C. The revocation boundary

Let t_c be the last policy check and t_s the provider send. A revocation that removes a required right and is visible at t_c blocks the action. A revocation between t_c and t_s cannot be foreseen at t_c. Disabling a cache does not make these events atomic.

The fresh-check baseline has no transaction joining policy reads and admission. Its guarantee is limited to the state observed at the check. The proposed protocol instead validates policy versions while committing a send permit. Revocation updates those versions under the same ordering rule. A mismatch forces a fresh decision. The next section defines the storage and timing rules needed for that claim.

A permit defines a local ordering point. It does not ensure that a provider receives no request after a later revocation. That stronger property requires authority validation at provider or gateway acceptance. A local check cannot recall a network packet or an accepted write.

VI. A Version-Checked Admission Protocol

A. State and allowed transitions

Let σ be the committed authority state and $v(\sigma) = (v_o, v_p)$ its organization and platform epoch pair. An epoch names a state change, not a duration. Define $\mathrm{Allowed}(\sigma, I, x, t)$ as the ceiling, path, approval, retry, and deadline predicates for the exact action x. Each predicate must have a known true answer. The state also stores worker generation g, normalized action x, and call state q. A new admission changes q from intent to admitted exactly once for that attempt.

TABLE II
The authority protocol has distinct admission and replay operations. The outside provider has a separate state.

Event	Guard	State change
Authority write	Lock each changed epoch.	Write policy and increment epochs in one commit.
Restore	Saved identity and parent links remain valid.	Use a new generation; unresolved admitted work is unknown.
Admit	Fresh versions, exact identity and action, live generation and deadlines.	Intent becomes admitted with its permit.
Save result	The result names its admitted attempt.	Store immutable completion evidence.
Replay	Current replay access and exact saved input.	Return stored output; no provider send.

B. Version validation

The proposed storage model has an organization epoch and a platform epoch. An epoch is a counter that never decreases or repeats. Every change to member, role, token, resource attributes, channel, bot, delegation, or approval state increments the organization epoch in the same transaction. Changes to platform rules or the resource catalog increment the platform epoch. Additions and deletions count as changes. This broad coverage avoids missing a newly added deny or a removed grant. Policy meaning is fixed during an evaluation run.

An execution stores its immutable identity, accepted input route, parent links, and current worker generation. Each call stores exact normalized action parameters, an attempt number, any approval or retry decision, and its state: intent, admitted, completed, or unknown. A result is unknown when an admitted write may have occurred but no result is known. Approval names the exact action and attempt it covers; it does not authorize changed input.

The admission procedure has four steps:

1) Read authority data and both epochs from one coherent snapshot. Check the whole path, ceiling, approval, and any retry decision maker. Record every relevant expiry. Task conditions use the stored actor; separate owner and decision-maker checks use those named principals. A failed or unknown check ends the procedure.
2) Start a transaction that takes shared locks on the platform epoch and then the organization epoch. Shared locks allow other admission readers but exclude a

writer. Then take exclusive locks on the execution and call records. Each authority writer takes an exclusive lock on every epoch it changes. All operations follow the same global lock order. Compare current epochs with the saved pair. A mismatch aborts admission and starts a fresh check.

3) Require the expected generation, an open execution, unchanged identity and action, and an intent for this attempt. In one guarded transition, read the trusted clock, require every deadline to be in the future, record a permit, and change intent to admitted. The permit binds identity, action, attempt, generation, epochs, and admission time. This transition is the local admission point t_a if its transaction commits; an aborted transaction has no admission point. Sending waits for commit. An expiry after t_a does not withdraw the permit. Thus expiry, like revocation, is ordered against admission and not provider acceptance.
4) Only the trusted sender can use that permit to make the bound request. It saves a known result as completed. A lost write result becomes unknown. On restart, an admitted call without a result is also unknown. A new send needs fresh admission; an old permit cannot replace current checks. Unknown writes require an explicit retry or stop decision.

Replay is a different operation. It checks current identity and path access, then returns only an already completed call with exactly matching parameters. A closed review may replay its own result but cannot advance. Missing membership or a revoked original token still blocks replay. Stop decisions cause no new send and can close unknown work without regranting rights.

The epoch protocol is deliberately coarse. An unrelated change in the same organization can force a retry. Shared epoch locks avoid an exclusive global lock for every admission, but a platform writer still waits for all overlapping admission readers. The final locked section must contain only local reads and writes, with no model or network call. Finer versions need a complete read set, including collection changes, to retain the proof. Clock expiry also needs a trusted admission-time check because time passes without an epoch change. A permit is evidence of local admission, not a guarantee of one remote effect or of continued rights at provider acceptance.

C. Cost and progress conditions

Let h be path depth, n the number of policy nodes read, and m the size of the finite action universe. A direct finite implementation checks the path in $O(h)$ record visits and evaluates one concrete action in $O(n)$ predicate work. Checking all positive scopes by enumeration takes $O(nm)$ work. An oracle for an unbounded condition language may have no complete decision procedure. The protocol requires sound refusal on an unknown answer, not a claim that all safe roles can be accepted.

An admission attempt takes two shared epoch locks and exclusive locks for the execution and call. Once policy and path evaluation are complete, validation uses a constant number of metadata checks. Comparing exact input bytes takes time in the size of that input. The cost of a failed version check includes another full evaluation. There is no finite retry bound when another process changes policy without end. Progress needs a period with no relevant write, a live eligible worker, a terminating evaluator, and fair lock service. These are progress assumptions; the safety proof does not need a successful admission.

VII. Analytical Claims and Failure Traces

Claim 1: restriction survives delegation. Assume a child preserves the actor and original token, validates its source path, and intersects its available actions with its parent's ceiling and its task restriction. Then a chain of children cannot gain an action outside the root ceiling at the same evaluation time. For one edge the claim is set inclusion. Applying that inclusion to each edge proves the claim by induction. The proof fails if any edge replaces the actor, drops the token, or treats approval as a union with extra authority. It also says nothing about historical rights unless the initial ceiling is retained.

Claim 2: the positive-scope guard is sufficient under a fixed binding. Let W be the writer's allowed set in the checked snapshot. Assume sound leaf matching and coverage. If the guard accepts a policy p, then its effective allowed set under that binding is a subset of W.

Proof. Use induction on the policy tree. An allow leaf is covered by the guard; a deny or no-answer leaf grants nothing. A rule list, alternative, or ordered choice can allow only through an allowing child. A conjunction can allow only when every child allows, so any one child is a witness. A ceiling can allow only when its base allows. The inductive hypothesis supplies a matching covered allow leaf in each case. Empty nodes never allow. Every granted action is therefore in W. The A-and-B example proves that the converse is false. The proof excludes a policy operator that grants by complement without a positive witness. It also excludes principal rebinding.

Claim 3: revocation before a fresh check blocks a new send. Assume the check observes the committed revocation, a revoked token gives no rights, failure stops the path, and every new send crosses the check. The send condition then has an empty token term, so no action can pass. This is a conditional safety result. A stale storage read or a tool that bypasses the boundary defeats its premises.

Claim 4: admission uses current authority at its local ordering point. Assume complete epoch coverage, coherent reads, sound evaluation, a trusted clock, and the stated reader and writer locks. Every committed admission of (I, x) satisfies $\text{Allowed}(\sigma_a, I, x, t_a)$, where σ_a is the authority state at its local admission point.

Proof. Let σ_r be the read snapshot and v_r its epochs. Evaluation established every non-time predicate on σ_r. If the current pair differs from v_r, admission aborts. If it is equal, no relevant write committed after that snapshot: a relevant write must increment one epoch, and epochs cannot repeat. The reader locks then exclude such a write until the guarded transition. The transition checks time, generation, exact input,

and call state under lock. The same predicates therefore hold at t_a. No effect is attributed to an aborted transaction. This proves the stated local property.

Corollary. A revocation that commits before t_a and leaves a required predicate false through t_a prevents admission. The same is true of a deadline reached by t_a. Neither result promises that rights remain valid at network send or provider acceptance. A later regrant may support a new admission. A policy write outside epoch coverage invalidates the proof.

Claim 5: replay adds no send authority. A replay can return only the result of an exactly matching completed call. Its transition contains no provider send and cannot change completed to intent. Thus any sequence of replay steps adds zero new admissions. This is a transition invariant, not proof that the original outside write happened only once. A retry is a separate attempt and must satisfy Claim 4 again.

The following traces are constructed examples. They report logical outcomes, not observed attack counts or timing measurements.

TABLE III
A REVOCATION TRACE WITH A SAVED APPROVAL. THE ORDERING IS ILLUSTRATIVE.

Point	State or event	Required result
1	Speaker asks a channel bot for work.	Keep owner O and actor S distinct.
2	A bot makes helper task D.	Child keeps S and token K.
3	Approver accepts the tool action.	Save the choice; keep the ceiling.
4	K is revoked; worker stops.	Stored approval gains no rights.
5	New worker restores D.	Reload K and refuse a new send.

The channel trace retains the speaker despite a change of worker. After K is revoked, its empty permission set blocks every new action. An approval cannot replace that set.

A second trace shows why the send boundary needs care. A token is valid at the final read. Another transaction revokes it. The worker then marks the call as sending and contacts the provider. Both statements, "every send had a recent check" and "a request was sent after revocation," can be true. Version validation can reject the permit if revocation won the local order. It cannot undo a permit that won the order first or a request already accepted elsewhere.

A third trace separates review completion from authority growth. A reviewer resolves a request, and the provider result is saved. The process stops before the chat checkpoint. On resume the review is closed. Rejecting all replay would lose the visible result of a completed action. Allowing all advance would reopen authority. The bounded solution permits replay of that saved call with unchanged input while refusing a new call. Revoking the original token remains a separate reason to stop access.

VIII. Synthetic Fixtures and Defined Baselines

A minimal fixture has organization G, owner O, speaker S, computers X and Y, and token K. Both people are active and have the required channel and delegation rights; K permits those route actions. For computer use, O has X and Y, S has only X, and K permits only X. An enabled channel belongs to O and accepts S. Its bot and helper impose no further restrictions. The child keeps S, K, and that parent path. The platform denies nothing. Thus the child can use X and cannot use Y.

Define four weakened baselines. **Owner substitution** replaces actor and token with the owner's session at send time, admitting Y. **Token dropping** keeps S but removes K on restart. Giving S both computers while keeping K limited to X exposes that fault: only the weakened baseline admits Y. **Admission-only checking** stores the ceiling at input and checks later actions against that saved set. Revoking K after input makes its next send unsafe. **Approval substitution** allows an approved action without current-rights intersection. Approval before revocation then permits an action the ceiling rejects.

These baselines are definitions, not reports about software. The fresh-check baseline differs from admission-only checking: it reloads rights before every send but does not validate their versions at admission. The two role guards differ only in whether they compare the effective set or all positive leaf scopes. Keeping these changes separate lets a reproduction attribute each counterexample to one missing rule.

IX. Reproducible Finite Study

The Python model in `papers/experiments/papers_01_05.py` is separate from the product. Its action universe is the three named resources X, Y, and Z. It compares set intersection with a per-resource Boolean oracle. It also compares effective-set and positive-scope guards on flat allow/deny policies. The schedule model records local admission and network send as different events. An action can be valid at admission and sent after revocation; this is counted as a later send, not an unsafe admission.

The extended checks are in `papers/experiments/paper_02_extended.py`. The reproduction command is `devenv shell -- python3 papers/experiments/run.py --seeds 30 --output .cache/e2e/papers.json`. The runner records source hashes, seed values, explored bounds, exact counts, and limits. This run passed 29,356 checks for this paper. That total includes 30 seeded fixtures as well as the exhaustive cases below. Repeated assertions on one state are not independent trials.

TABLE IV
EXACT MODEL COUNTS. THE ROLE COUNTS COVER FLAT DENY-PRIORITY POLICIES UNDER A FIXED PRINCIPAL BINDING.

Finite family	Cases	Observed result
Four rights sets over three resources	4,096	No extra right; per-resource oracle agrees.
Writer, allow, and deny sets	512	343 safe final sets; 216 pass the leaf guard.
Safe final set refused by leaf guard	127	All have a wider positive scope.
One revocation around read, admit, and send	4 orders per policy	Separate read admits one invalid action; epoch validation admits none.

There are eight subsets of the three-resource universe. The rights study checks all 8^4 four-limit tuples. The role study checks all 8^3 writer/allow/deny tuples. Among the 343 safe final policies, 127 are refused by the leaf guard. This is a measured cost of the guard's chosen contract in this finite family. It is not an estimate of the refusal rate for user policies. The guard accepts no unsafe policy in this family, as Claim 2 predicts.

Let R be the read, A the local admission, S the send, and V the revocation. The schedule study explores all four placements of V with R before A before S. It uses the same order for three decision rules. The separate-read baseline carries the answer from R to A. An atomic authority oracle reads the live predicate at A itself. The epoch rule compares the version read at R with the version at A. The oracle is an ideal reference, not an implementation that closes a race merely by reading recently.

For R,V,A,S, the separate-read baseline admits an invalid action. The atomic oracle and epoch rule refuse it. For R,A,V,S, all three rules admit while rights are valid and send after revocation. Across the four orders, the separate-read baseline has one invalid admission and two sends after revocation. The other rules have zero invalid admissions and one later send each. Thus a count of later sends alone would wrongly classify a valid local ordering as a protocol failure.

Five further checks accept an unchanged permit before expiry and refuse it at expiry or after a change of version, generation, or action. The expiry check holds versions fixed. It establishes why version comparison alone cannot cover the passage of time. These checks test the model predicate; they do not test a database clock or lock implementation.

The finite study can find a wrong set or schedule rule inside these bounds. It does not run a policy parser, database lock, process restart, or provider. It checks flat-policy instances of Claim 2 and a single-write abstraction of Claim 4. Claims about full parent paths, all policy operators, and replay still need their stated proof premises and separate product tests.

X. PRODUCT EVALUATION PROTOCOL

The remaining product study should answer three separate questions: whether scope checks match their stated semantics, whether every resumed action retains identity, and where revocation takes effect. These questions need different oracles. An oracle is an independent method for deciding the expected answer.

For scope checking, generate a finite universe with several organizations, owners, resource kinds, and actions. Enumerate policy truth on every resource value as the semantic oracle. Generate nested policy trees with wildcards, denies, empty conditions, ordered branches, and principal references. Compare final-set inclusion with positive-leaf inclusion. The two are deliberately different. Record accepted unsafe scopes, refused safe final policies, and cases where coverage cannot prove an answer. Keep fixed-principal and rebound-principal cases in separate groups. A finite model checks the generated universe; it does not prove all numeric or string domains.

For identity preservation, run direct, channel, task, and review paths with both sessions and restricted tokens. Insert a pause before child creation, after approval, after intent, before sending, and before checkpoint. At each pause change one of the actor, token, owner membership, bot state, channel state, delegation link, or decision maker's rights. Resume on a new worker. Record the full identity at the authorization boundary and the provider's accepted calls. A mismatch between the expected actor/token pair and the observed pair is a failure even if the chosen test action happened to remain allowed.

Run the four weakened baselines and the fresh-check baseline on identical inputs. Compare both role guards to measure conservative rejection as well as safety. Further ablations remove parent-path validation, fresh capability answers, and the decision-maker recheck one at a time.

For revocation, use controlled transaction barriers instead of wall-clock sleeps. Force revocation before policy read, after the read but before admission, and after admission but before provider acceptance. Compare the fresh-check baseline with the proposed permit protocol. A provider stub should record acceptance independently of application logs. Keep lost write replies distinct from refused sends; an unobserved effect is not proof that no action occurred.

Report unsafe admissions per attempted schedule, identity mismatches, conservative rejections, and fresh-check storage reads. Report latency as distributions for stable-rights work and revocation-heavy work. Use the same generated seeds and schedules for paired comparisons. Keep correctness counts exact. If randomized tests observe no failure, state the number and sampling scheme; do not convert that result into a universal security claim. Confidence intervals for sampled failure rates and latency should be computed across independent runs, with shared seeds treated as paired data. Exhaustive small-state results should instead state the explored bounds.

For each result, archive the normalized action, original actor and token references, parent path, policy snapshot and epochs, call state, admission time, and independent provider acceptance record. Use nonsecret test identifiers. The reference checker reads this trace and the controlled policy history; it must not call the authorization function under test. Publish all failed schedules as well as totals. A provider acceptance with no matching admission is a boundary bypass; a matching admission with a false reference predicate is an admission failure. These are different faults.

The evaluation must retain the difference between local admission and provider-side cancellation, even if all selected cases pass. A passing finite test set cannot remove the stated timing and storage assumptions.

XI. Limits and Conclusion

This model assumes sound policy coverage, trusted identity storage, and complete placement of checks. It does not verify a policy library, adapters, or external services. Separately reading several policy records does not satisfy the coherent-snapshot premise. Parameter restrictions also require tools to present the real action to the boundary. A check of one destination does not protect a different destination chosen inside a tool.

The finite model covers small sets, flat deny-priority policies, and one revocation. It does not establish the structural induction for every parser operator or the lock premise of Claim 4. Product tests can test these premises but cannot turn bounded exploration into an unbounded proof. No hardware, latency, throughput, or field security result is claimed.

A complete implementation must trace every send path, including background work and notices. Fixed creation-time ceilings and provider-checked permits are distinct extensions. The proposed epoch protocol establishes a local ordering only under its stated writer and sender rules. Retries, worker takeover, and provider idempotency need their own analysis of repeated effects.

Authority preservation requires more than remembering who started a chat. The resource owner, actor, original token, bot, and delegation each have a distinct role. Positive-scope checks can bound new policy grants under a clear principal binding. Current checks can stop resumed work after rights are removed. Separate replay from advance allows known results to be recorded without creating a new right to act. With explicit time and storage assumptions, these mechanisms support precise, testable claims about multi-principal delegation while leaving the external revocation boundary visible.

References

[1] N. Hardy, "The Confused Deputy (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988, doi: 10.1145/54289.871709.

[2] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.

[3] B. Lampson, M. Abadi, M. Burrows, and E. Wobber, "Authentication in Distributed Systems: Theory and Practice," *ACM Transactions on Computer Systems*, vol. 10, no. 4, pp. 265–310, 1992, [Online]. Available: https://bwlampson.site/45-AuthenticationTheoryAndPractice/Acrobat.pdf

[4] M. Abadi, M. Burrows, B. Lampson, and G. Plotkin, "A Calculus for Access Control in Distributed Systems," *ACM Transactions on Programming Languages and Systems*, vol. 15, no. 4, pp. 706–734, 1993, [Online]. Available: https://homepages.inf.ed.ac.uk/gdp/publications/Calculus_for_Access_Control.pdf

[5] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud," in *Network and Distributed System Security Symposium*, 2014. doi: 10.14722/ndss.2014.23212.

[6] D. Hardt, "The OAuth 2.0 Authorization Framework," Request for Comments 6749, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749.html

[7] M. B. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, "OAuth 2.0 Token Exchange," Request for Comments 8693, Jan. 2020. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8693.html

[8] J. Park and R. Sandhu, "The UCONABC Usage Control Model," *ACM Transactions on Information and System Security*, vol. 7, no. 1, pp. 128–174, 2004, doi: 10.1145/984334.984339.

[9] R. Pang *et al.*, "Zanzibar: Google's Consistent, Global Authorization System," in *2019 USENIX Annual Technical Conference*, USENIX Association, 2019, pp. 33–46. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/pang

[10] J. W. Cutler *et al.*, "Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization (Extended Version)." [Online]. Available: https://arxiv.org/abs/2403.04651

[11] C. Disselkoen *et al.*, "How We Built Cedar: A Verification-Guided Approach." [Online]. Available: https://arxiv.org/abs/2407.01688